SMARTROAD S.O.S. | EVIDENCIA DE ENTREGA
CRUD Mes 1 / Sprint 2 - Node.js + Express + MySQL

Este paquete queda local y no se sube automaticamente. Cada companero debe crear su rama y completar las evidencias

1. ALCANCE
El CRUD singular esta integrado en /home/oswal/SmartRoadSOS/backend sin eliminar las rutas plurales existentes.
Repositorio central: https://github.com/bandreswscanga-debug/SmartRoad
Cadena: conductor -> vehiculo -> dispositivo -> evento -> alerta.
API local: http://127.0.0.1:4000
phpMyAdmin local: http://127.0.0.1:8080
Base de datos: smartroad_sos

2. REPARTO
Parte 1 - Companero/a 1
Rama: evidencia/parte-1-colaboracion-cruds
Responsabilidad: repositorio, historial de commits, Postman y CRUD funcional.

Parte 2 - Companero/a 2
Rama: evidencia/parte-2-persistencia-scrum
Responsabilidad: persistencia MySQL y tablero Scrum.

3. PARTE 1 - COLABORACION Y CRUD
3.1 Repositorio
Completar la URL real de GitHub o GitLab y los nombres de los integrantes.
Tomar una captura de History/Commits filtrada al ultimo mes.
Deben verse autores distintos, fechas y mensajes.
Archivo sugerido: 01-git-historial-ultimo-mes.png

3.2 Preparacion
cd /home/oswal/SmartRoadSOS/backend
pnpm install
pnpm db:setup
pnpm start
Comprobar: curl http://127.0.0.1:4000/api/health

3.3 Postman
Importar:
Entregables/evidencia-crud-mes1/postman/SmartRoad-SOS-CRUD.postman_collection.json
Entregables/evidencia-crud-mes1/postman/SmartRoad-SOS-Local.postman_environment.json
La coleccion contiene 29 solicitudes y las cinco operaciones CRUD para las cinco entidades.
Orden: crear conductor, vehiculo, dispositivo, evento y alerta; consultar; actualizar; validar; eliminar en orden inverso.

Endpoints:
GET /api/conductor POST /api/conductor
GET /api/conductor/:id PUT /api/conductor/:id
DELETE /api/conductor/:id
Repetir para /api/vehiculo, /api/dispositivo, /api/evento y /api/alerta.

Validaciones esperadas:
- Campo NOT NULL faltante: 400
- FK inexistente: 400
- Registro inexistente: 404
- Duplicado o relacion 1:1 violada: 409
- Creacion correcta: 201
- GET, PUT y DELETE correctos: 200

Capturas sugeridas:
02-postman-get-conductor.png
03-postman-post-conductor.png
04-postman-put-conductor.png
05-postman-delete-conductor.png
06-postman-vehiculo-join.png
07-postman-fk-invalida-400.png
08-postman-cadena-completa.png

4. PARTE 2 - PERSISTENCIA Y SCRUM
4.1 Persistencia
Abrir http://127.0.0.1:8080 o DBeaver.
Seleccionar smartroad_sos y la tabla conductor.
Tomar una captura antes de insertar: 10-db-conductor-antes.png.
Ejecutar en Postman el POST de conductor con un valor unico, por ejemplo EV260924A.
Tomar una captura de la misma tabla despues: 11-db-conductor-despues.png.
El documento, nombre y demas datos deben coincidir exactamente con la peticion Postman.
Si se creo la cadena completa, capturar tambien vehiculo, dispositivo, evento y alerta.

4.2 Scrum
Abrir Jira, Trello o GitHub Projects real.
Capturar el tablero con la columna Terminado/Done visible.
Deben aparecer terminadas las historias de configuracion base, login y CRUD de Semanas 1 a 4.
Archivo sugerido: 13-scrum-tablero-terminado.png

5. RAMAS Y COMMITS
git switch -c evidencia/parte-1-colaboracion-cruds
git switch -c evidencia/parte-2-persistencia-scrum
git status
git add Entregables/evidencia-crud-mes1
git commit -m "evidencia: agrega pruebas funcionales CRUD"
No incluir .env ni contrasenas. Este entorno no ejecuta push.

6. CHECKLIST FINAL
[ ] URL real del repositorio y commits de varios integrantes.
[ ] Postman demuestra GET, POST, PUT y DELETE.
[ ] FK invalida y campos faltantes devuelven 400.
[ ] JOIN de vehiculo visible.
[ ] Captura de base antes y despues.

[ ] Dato Postman identico al dato persistido.
[ ] Tablero Scrum con historias en Terminado.
[ ] Commits realizados en las ramas de cada companero.

Archivos fuente: PARTE-1-COLABORACION-Y-CRUD.md, PARTE-2-PERSISTENCIA-Y-SCRUM.md, INSTRUCCIONES-R